Лабораторная работа №7. Коллективные операции в MPI.

1. Что вычисляет программа и как распределяются данные
   Программа вычисляет сумму элементов массива $a_i = i + 1$ размером N. Процесс 0 инициализирует массив и рассылает размер N всем процессам с помощью MPI_Bcast. Затем массив распределяется между процессами с помощью MPI_Scatterv (это позволяет работать с любым N, даже если оно не делится на число процессов P). Каждый процесс считает сумму своей части, после чего результаты собираются тремя способами: через MPI_Gather, MPI_Reduce и MPI_Allreduce.

2. Компиляция: mpicc mpi_array_sum.c -o mpi_array_sum.exe
   Запуск: mpiexec -n 4 ./mpi_array_sum.exe 1000001

3. Результаты проверки корректности:

| Число процессов P | Размер массива N | Ожидаемая сумма | Gather | Reduce | Allreduce | Корректно? |
|---|---|---|---|---|---|---|
| 2 | 10 | 55 | 55 | 55 | 55 | Да |
| 4 | 15 | 120 | 120 | 120 | 120 | Да |
| 8 | 1003 | 503506 | 503506 | 503506 | 503506 | Да |
| 4 | 1 000 001 | 500001500001 | 500001500001 | 500001500001 | 500001500001 | Да |
| 4 | 100 000 000 | 5000000050000000 | 5000000050000000 | 5000000050000000 | 5000000050000000 | Да |

4. Результаты экспериментов:
   Эксперимент 1. Влияние размера массива (P=4):

| Размер N | Время, с |
|---|---|
| 10 000 | 0.000256 |
| 100 000 | 0.000246 |
| 1 000 000 | 0.000355 |
| 10 000 000 | 0.000359 |

   Эксперимент 2. Влияние числа процессов (N=1 000 000):

| Число процессов P | Время, с | Ускорение $S_P$ | Эффективность $E_P$ |
|---|---|---|---|
| 1 | 0.000021 | 1.00 | 1.00 |
| 2 | 0.000200 | 0.105 | 0.05 |
| 4 | 0.000321 | 0.065 | 0.02 |
| 8 | 0.001001 | 0.021 | 0.003 |

   Эксперимент 3. Сравнение Gather и Reduce:

| Число процессов P | Способ агрегирования | Число повторений | Среднее время операции, с | Результат корректен? |
|---|---|---|---|---|
| 2 | Gather + сумма на процессе 0 | 10000 | 0.000000217 | Да |
| 2 | Reduce | 10000 | 0.000000263 | Да |
| 4 | Gather + сумма на процессе 0 | 10000 | 0.000034802 | Да |
| 4 | Reduce | 10000 | 0.000033466 | Да |
| 8 | Gather + сумма на процессе 0 | 10000 | 0.000119494 | Да |
| 8 | Reduce | 10000 | 0.000158915 | Да |

5. Вывод:
   В ходе работы подтверждено назначение коллективных операций: MPI_Gather собирает локальные значения от всех процессов в массив на корневом процессе; MPI_Reduce сразу применяет к ним операцию (например, MPI_SUM) и возвращает результат только корневому процессу; MPI_Allreduce делает то же самое, но возвращает итог всем процессам коммуникатора. Результаты экспериментов показали, что при увеличении размера массива от 10 тысяч до 10 миллионов время выполнения почти не меняется (остаётся в пределах 0.0002–0.0004 с), так как операции сложения выполняются процессором за доли миллисекунды. При увеличении числа процессов время, наоборот, растёт (с 0.000021 с на 1 процессе до 0.001001 с на 8 процессах), а ускорение падает ниже единицы. Это объясняется тем, что для такой лёгкой вычислительной задачи накладные расходы на пересылку сообщений (Bcast, Scatterv, Gather, Reduce) начинают доминировать над полезной работой. Это наглядно демонстрирует, что параллелизм в MPI оправдан только для задач с высокой вычислительной сложностью, где время расчётов превышает время обмена сообщениями.